Birla Vishvakarma Mahavidyalaya (BVM) Engineering College
Vallabh Vidyanagar, Anand, Gujarat

Department of Electrical Engineering

ESP32 Bluetooth A2DP Speaker with OLED User Interface

Design, Hardware, Firmware and Verification Report

Submitted by

Name	Enrolment No.
Parmar Dev	23EE080
Parmar Vishant	25EE303
Patel Jay	25EE302

Programme: B.Tech. Electrical Engineering

Project platform: ESP32-WROOM-32 · Arduino core · I^2S audio · 128×64 OLED

Source repository: `github.com/devoltamp/mp3-player`

Academic Year 2026–27

Contents

Executive Summary 1

1 Introduction 2
1.1 Motivation . . . 2
1.2 Objectives and requirements . . . 2
1.3 Repository inventory . . . 2

2 System Architecture 2
2.1 Design choices . . . 3

3 Hardware Design 3
3.1 Bill of materials . . . 3
3.2 Power path . . . 4
3.3 Audio output stage . . . 4
3.4 Schematic review . . . 4
3.5 GPIO allocation . . . 4

4 Firmware Design 5
4.1 Toolchain and dependencies . . . 5
4.2 Software architecture . . . 5
4.3 Bluetooth, audio and metadata . . . 6
4.4 Input handling . . . 7
4.5 User interface . . . 9
4.6 Battery gauge . . . 9
4.7 Display power management . . . 10
4.8 Persistence . . . 10
4.9 Concurrency and robustness . . . 11

5 Verification Plan 11

6 Design Review Findings and Recommendations 11

7 Conclusion and Future Work 11

Appendix – Full Schematics 13

Executive Summary

This project builds a battery-powered *Bluetooth audio receiver* (A2DP sink) around an ESP32-WROOM-32 module. A phone streams music over Bluetooth Classic; the ESP32 decodes the SBC stream, forwards 16-bit/44.1 kHz stereo PCM over I^2S to a class-D amplifier (MAX98357A) driving a speaker, and renders a rich user interface on a 1.3" 128×64 OLED. The repository is named `mp3-player`, but the firmware implements a *streaming speaker*, not local file playback; the SD-card socket drawn in the schematic is reserved for future work and is not accessed by either sketch.

Headline results.

- A complete, readable firmware (788 lines, `main.ino`) with rotary-encoder control, AVRCP remote control and metadata, marquee text, animated equaliser, battery gauge with LiPo curve, auto-dim and burn-in protection, and flash-persisted settings.
- A second variant (`updated.ino`, 739 lines) replacing the encoder by four tactile buttons.

- Two hardware variants: speaker output (MAX98357A) and headphone output (PCM5102A DAC with 3.5 mm jack), each with TP4056 charging.
- A set of candid review findings (Section 6) – chiefly pin-map and part-number mismatches between schematic and firmware – that should be closed before a PCB is ordered.

1 Introduction

1.1 Motivation

Commodity Bluetooth speakers hide their internals. Building one from an ESP32 exposes every layer of an embedded audio product: radio stack, real-time audio streaming, analogue power management, human interface design and non-volatile configuration. The author's stated aim (repository README) is a "potato iPod" – a small, cheap, hackable audio device.

1.2 Objectives and requirements

Table 1 lists the requirements reverse-engineered from the firmware header and README, with the section in which each is addressed.

Table 1: Requirements derived from the repository.

ID	Requirement	Section
R1	Pair with any phone as an A2DP sink; auto-reconnect to the last host	4.3
R2	Stereo 16-bit / 44.1 kHz output over I^2S	3.3
R3	Local control: volume, play/pause, next, previous, page switch	4.4
R4	Display title, artist, album, elapsed/total time, progress	4.5
R5	Run from a single LiPo cell with USB charging and a battery gauge	3.2, 4.6
R6	Protect the OLED against burn-in; minimise idle power	4.7
R7	Persist volume and brightness across power cycles	4.8
R8	Reproducible build from bundled libraries	4.1

1.3 Repository inventory

Table 2: Contents of the repository (git history: 11 commits).

Path	Description
`src/main.ino`	Reference firmware, rotary-encoder edition (788 lines)
`src/updated.ino`	Four-button edition with placeholder "Live Lyrics" page (739 lines)
`hardware/mp3.kicad_sch`	Schematic, MAX98357A speaker output (KiCad 10)
`hardware/with headphone.kicad_sch`	Schematic, adds PCM5102A DAC and audio jack
`libraries/*.zip`	Adafruit BusIO 1.17.4, GFX 1.12.6, SSD1306 2.5.16; arduino-audio-tools 1.2.6; libhelix 0.9.4; ESP32-A2DP 1.8.11
`README.md` (+ sub-READMEs)	Usage notes, library install steps, schematic screenshots

2 System Architecture

Figure 1 shows the system partitioned into radio, processing, audio, power and human-interface domains. The ESP32 runs the Bluetooth host and controller on its two Xtensa cores (managed by the A2DP library on FreeRTOS tasks), while the Arduino `loop()` cooperatively services the UI at up to 25 fps.

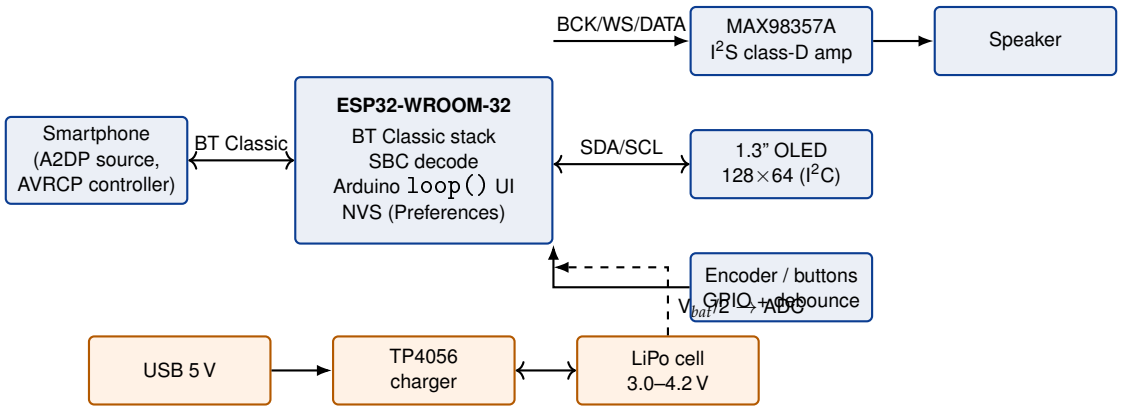


Figure 1: System block diagram. Orange blocks are the power path; the dashed line is the battery-voltage sense path used by the gauge (divider not shown in the schematic, see Section 6).

2.1 Design choices

Software I²S sink via audio-tools.
`I2SStream` is the output of `BluetoothA2DPSink`; no ring buffer code is hand-written, which removes the most common source of glitches.

Class-D amplifier with digital input.
The MAX98357A accepts I²S directly, so no DAC, filter or analogue gain stage is required; one 100 kΩ resistor selects the channel mode.

U8g2 full-buffer mode.
The `_F_` constructor keeps a 1 KiB frame buffer in RAM, allowing flicker-free compositing of popups and marquees.

Polling UI, ISR only for the encoder.
All non-time-critical logic is polled from `loop()`, which keeps state machines easy to read and test.

3 Hardware Design

3.1 Bill of materials

Table 3: Bill of materials extracted from the two schematics.

Ref.	Part	Function	Variant
U3	ESP32-WROOM-32	MCU + Bluetooth/Wi-Fi module	both
U1	MAX98357A	I²S class-D mono amplifier	both
U5	PCM5102A	I²S stereo DAC (line/headphone output)	headphone
U4	TP4056-42-ESOP8	Li-ion linear charger (4.2 V)	both
BT1	Battery cell	Single LiPo cell	both
U2	ER-OLEDM0.91-1x-I2C (symbol)	OLED display (fitted part: 1.3" module)	both
sd_card	SD socket (6-pin)	Reserved for local playback	both
SW1–SW6	Tactile push-button	Six GPIO inputs to GND	both
R1	100 kΩ	MAX98357A `SD_MODE` bias to 3.3 V	speaker
C1, C2	2.2 µF	PCM5102A charge-pump flying/decoupling	headphone
C3	0.5 µF (as drawn)	PCM5102A LDO output decoupling	headphone
J1	Audio jack (stereo, ground)	Headphone output	headphone
LS1	Speaker	4–8 Ω transducer	speaker

3.2 Power path

The TP4056 module is fed from +5 V (USB) and charges the cell BT1 through its BAT pin. In the schematic the battery is connected only to the charger; the +5 V net feeding the MAX98357A and the 3.3 V net feeding the ESP32 are drawn as power symbols. A complete product therefore still needs a load-sharing/boost stage and a 3.3 V regulator (or a module such as a charge-boost board) – see finding F5. Figure 2 shows the charging section as rendered from the KiCad file.

3.3 Audio output stage

Speaker variant. I²S signals `IO27` (DIN), `IO26` (BCLK) and `IO25` (LRCLK) drive the MAX98357A. With 16-bit stereo at $f_s = 44.1\,\mathrm{kHz}$ the bit clock is

$$f_{BCLK} = f_s \times N_{ch} \times N_{bit} = 44\,100 \times 2 \times 16 = 1.4112\,\mathrm{MHz}. \tag{1}$$

The amplifier's `GAIN_SLOT` pin is tied to ground and `SD_MODE` is pulled to 3.3 V through R1 = 100 kΩ. Per the MAX98357A datasheet, `SD_MODE` selects shutdown/left/right/mono-mix according to its voltage and `GAIN_SLOT` selects one of five fixed gains; the author should confirm the intended mode and gain against the datasheet tables for the supply used (5 V on VDD, 3.3 V on the bias).

Headphone variant. The PCM5102A is used in its PLL-clocked mode (`SCK` tied to ground), with `FLT`, `DEMP` and `FMT` strapped, 2.2 µF capacitors on the charge pump and 0.5 µF on the LDO output. Its ground-referenced output removes the need for DC-blocking capacitors at the jack, which is why it is the better choice for earphones (at higher cost, as noted in the README).

3.4 Schematic review

Both schematics were rendered directly from the `.kicad_sch` files (the README images are screenshots and are not reproduced here). Figure 8 (landscape page) shows the speaker variant in full; Figure 3 shows the three functional blocks at readable size, and Figure 9 the headphone variant.

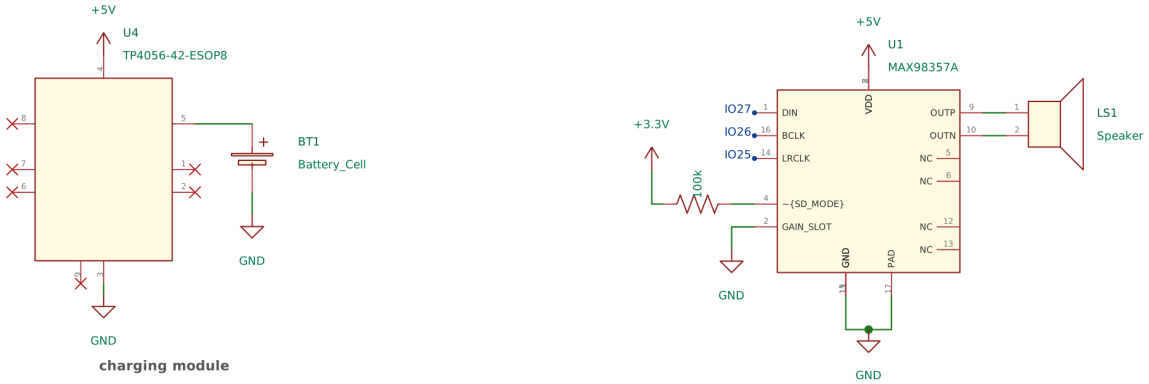


Figure 2: Left: TP4056 charging section with battery BT1. Right: MAX98357A amplifier with speaker LS1, 100 kΩ `SD_MODE` resistor and ground-strapped `GAIN_SLOT`.

3.5 GPIO allocation

Table 4 consolidates the pin usage from both sketches and the schematic. The disagreements between these sources are discussed in findings F1–F3.

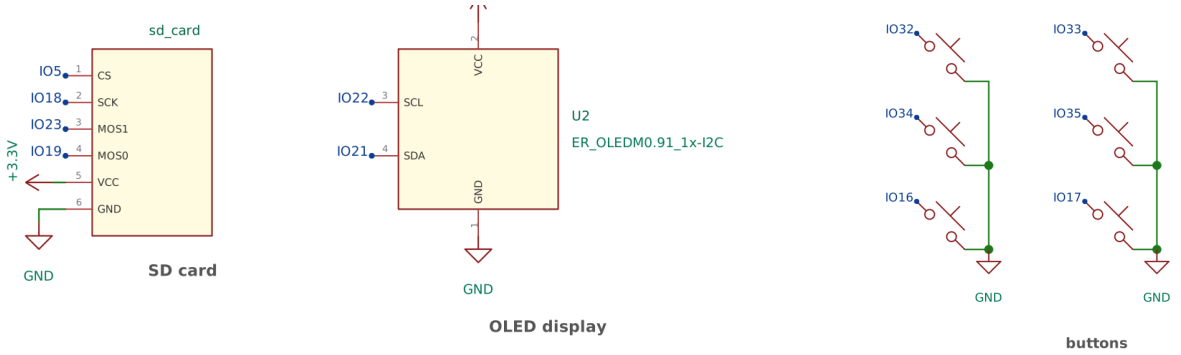


Figure 3: Left: SD-card socket (SPI via labels `IO5`, `IO18`, `IO23`, `IO19`) and OLED (I^2C `IO21/IO22`). Right: two groups of three tactile buttons with a common ground rail.

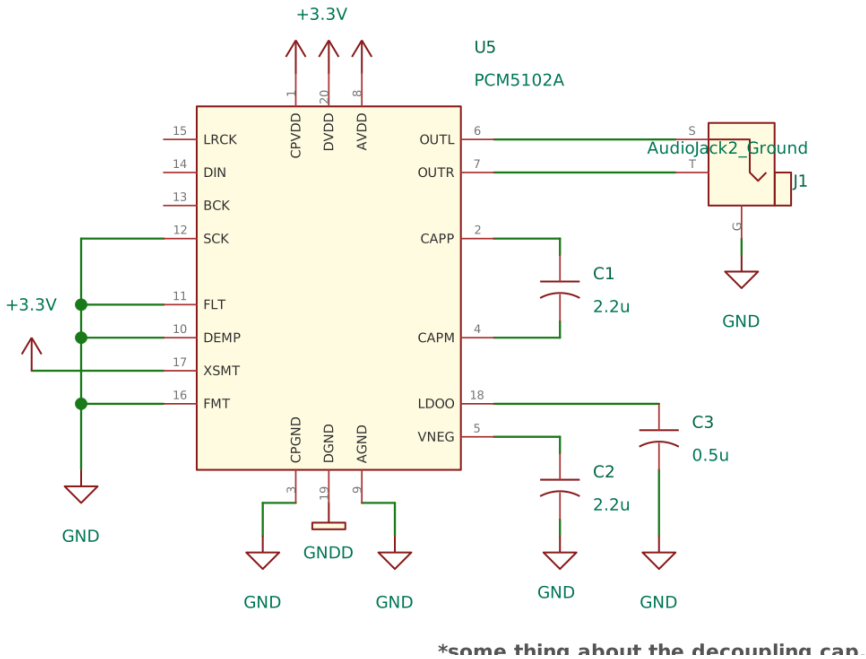


Figure 4: Headphone variant: PCM5102A DAC with decoupling capacitors C1–C3 and audio jack J1.

4 Firmware Design

4.1 Toolchain and dependencies

The sketch targets the Arduino-ESP32 core and uses **U8g2** (display), **ESP32-A2DP** and **arduino-audio-tools** (Bluetooth sink and I^2S output). The `libraries/` folder bundles pinned ZIP archives (installation order is numbered 01–06 in the README). Two practical notes: the header comment of `main.ino` states that Adafruit GFX/SSD1306 are *no longer needed*, so archives 01–03 are legacy; and U8g2 is not bundled and must be installed from the Library Manager.

4.2 Software architecture

The program is a cooperative super-loop (Listing 1) of small, non-blocking service functions; each one keeps its own timestamps and returns immediately if there is nothing to do. Cross-context data (encoder ticks, Bluetooth metadata) is exchanged through spin-lock-protected variables.

Listing 1: Main loop and display scheduling (`main.ino`, lines 771–788).

```
771 void loop() {
772   handleButton();
```

Table 4: GPIO map: firmware versus schematic.

Function	`main.ino` (encoder)	`updated.ino` (buttons)	Schematic
I²S BCK / WS / DATA	26 / 25 / 27	26 / 25 / 27	26 / 25 / 27
OLED SDA / SCL	21 / 22	21 / 22	21 / 22
Encoder CLK / DT / SW	32 / 33 / 14	–	–
Play-pause / Next / Prev / Page	–	14 / 12 / 13 / 4	–
Six push-buttons	–	–	32, 33, 34, 35, 16, 17
Battery sense (ADC1)	34	34	(not drawn)
SD card CS / SCK / MOSI / MISO	–	–	5 / 18 / 23 / 19

```
  handleEncoder();
  updateMetadata();
  updateBattery();
  updatePlaybackState();
  syncRemoteVolume();
  updateDisplayPower();
  savePrefsIfNeeded();

  static uint32_t lastDraw = 0;
  const uint32_t now = millis();
  const uint32_t frameMs = (wasPlaying || activeUntil(noticeUntil) || activeUntil(volumePopupUntil)) ?
      FRAME_MS_ACTIVE : FRAME_MS_IDLE;
  if (oledOK && !displayOff && now - lastDraw >= frameMs) {
    lastDraw = now;
    renderFrame();
  }
}
```

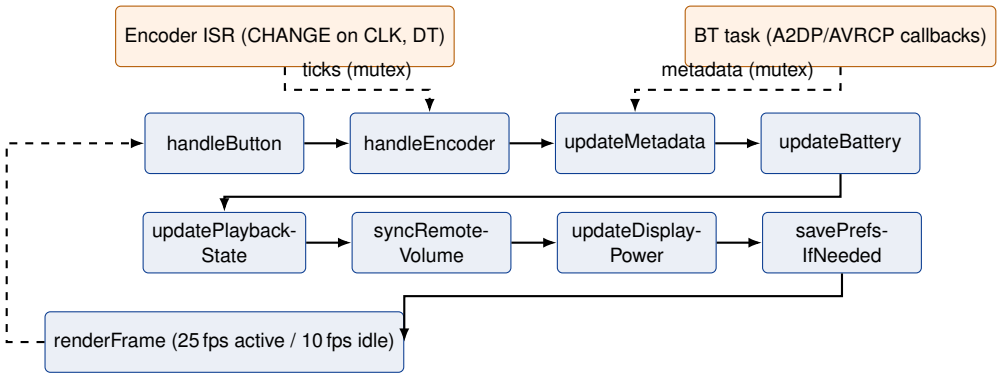


Figure 5: Execution model: one cooperative loop plus two asynchronous producers.

4.3 Bluetooth, audio and metadata

Initialisation (Listing 2) configures the I²S stream (44.1 kHz, 16-bit, stereo, 8 buffers of 512), enables auto-reconnect, requests AVRCP title/artist/album/playing-time attributes and starts the sink under the name `ESP32_BT_Speaker`.

Listing 2: Audio/Bluetooth initialisation (`main.ino`, lines 746–763).

```
  auto config = audioOutput.defaultConfig(TX_MODE);
  config.pin_bck = I2S_BCK;
  config.pin_ws = I2S_LRCK;
  config.pin_data = I2S_DATA;
  config.sample_rate = 44100;
  config.bits_per_sample = 16;
  config.channels = 2;
  config.buffer_count = 8;
  config.buffer_size = 512;
  audioOK = audioOutput.begin(config);
  if (!audioOK) showNotice("I2S failed", 3000);
```

```
757
758   bluetooth.set_auto_reconnect(true);
759   bluetooth.set_avrc_metadata_attribute_mask(
760     ESP_AVRC_MD_ATTR_TITLE | ESP_AVRC_MD_ATTR_ARTIST | ESP_AVRC_MD_ATTR_ALBUM |
          ESP_AVRC_MD_ATTR_PLAYING_TIME);
761   bluetooth.set_avrc_metadata_callback(onMetadata);
762   bluetooth.start(BT_NAME);
763   bluetooth.set_volume((volumePercent * 127 + 50) / 100);
```

Metadata arrives on the Bluetooth task. The callback copies strings into a pending buffer under a spin lock and sets a flag; the main loop swaps them out under the same lock (Listing 3) so that the display never reads a half-written string.

Listing 3: Metadata hand-off between Bluetooth task and UI (`main.ino`, lines 597–624).

```
597 void onMetadata(uint8_t attribute, const uint8_t *value) {
598   if (!value) return;
599   portENTER_CRITICAL(&metadataMux);
600   switch (attribute) {
601     case ESP_AVRC_MD_ATTR_TITLE: strlcpy(pendingTitle, (const char *)value, sizeof(pendingTitle)); break;
602     case ESP_AVRC_MD_ATTR_ARTIST: strlcpy(pendingArtist, (const char *)value, sizeof(pendingArtist));
          break;
603     case ESP_AVRC_MD_ATTR_ALBUM: strlcpy(pendingAlbum, (const char *)value, sizeof(pendingAlbum)); break;
604     case ESP_AVRC_MD_ATTR_PLAYING_TIME: pendingDurationMs = strtoul((const char *)value, nullptr, 10);
          break;
605     default: break;
606   }
607   metadataChanged = true;
608   portEXIT_CRITICAL(&metadataMux);
609 }
610
611 void updateMetadata() {
612   char title[sizeof(pendingTitle)], artist[sizeof(pendingArtist)], album[sizeof(pendingAlbum)];
613   uint32_t duration = 0;
614   portENTER_CRITICAL(&metadataMux);
615   const bool changed = metadataChanged;
616   if (changed) {
617     memcpy(title, pendingTitle, sizeof(title));
618     memcpy(artist, pendingArtist, sizeof(artist));
619     memcpy(album, pendingAlbum, sizeof(album));
620     duration = pendingDurationMs;
621     metadataChanged = false;
622   }
623   portEXIT_CRITICAL(&metadataMux);
624   if (!changed) return;
```

Titles may contain UTF-8; `displayText()` (lines 182–205) decodes code points, maps typographic punctuation to ASCII, keeps Latin-1 characters and replaces everything else with `?`, because the bundled bitmap fonts cover only that range. Elapsed time is not reported by the phone, so it is integrated locally while the stream state is `STARTED` and reset on a track change.

4.4 Input handling

Rotary encoder. Both encoder lines raise an interrupt on any edge. The ISR (lines 144–150) forms a 4-bit index from the previous and current two-bit Gray state and adds a $-1/0/+1$ entry from a 16-element table kept in DRAM, so the ISR is safe even when flash is busy. Four transitions make one detent (`ENC_TRANSITIONS = 4`); each detent changes volume by 4 %, so the full range is 25 detents.

Listing 4: Encoder decode and consumption (`main.ino`, lines 123–129 and 572–594).

```
572 void handleEncoder() {
573   portENTER_CRITICAL(&encoderMux);
574   const int steps = encoderTicks / ENC_TRANSITIONS;
575   encoderTicks -= steps * ENC_TRANSITIONS;
576   portEXIT_CRITICAL(&encoderMux);
577   const int movement = steps * ENC_DIRECTION;
578   if (!movement) return;
579   markActivity();
580
581   if (button.stable == LOW) {
```

```
582     // Press + rotate = brightness
583     button.rotated = true;
584     brightnessPercent = constrain(brightnessPercent + movement * 10, 5, 100);
585     displayDimmed = false;
586     applyContrast();
587     showNotice("Bright " + String(brightnessPercent) + "%", 900);
588     markPrefsDirty();
589     return;
590   }
591   const int previous = volumePercent;
592   volumePercent = constrain(volumePercent + movement * VOLUME_STEP, 0, 100);
593   if (volumePercent != previous) applyVolume(true);
594 }
```

Click gestures. One physical push-button yields five commands. The button is debounced for 25 ms; short presses are counted and resolved after a 300 ms quiet window (1 click = play/pause, 2 = next, 3 = previous); holding for 600 ms switches page; rotating while pressed adjusts brightness and suppresses the click on release. Consequently the play/pause action has a deliberate latency of about 325 ms. Figure 6 shows the state machine.

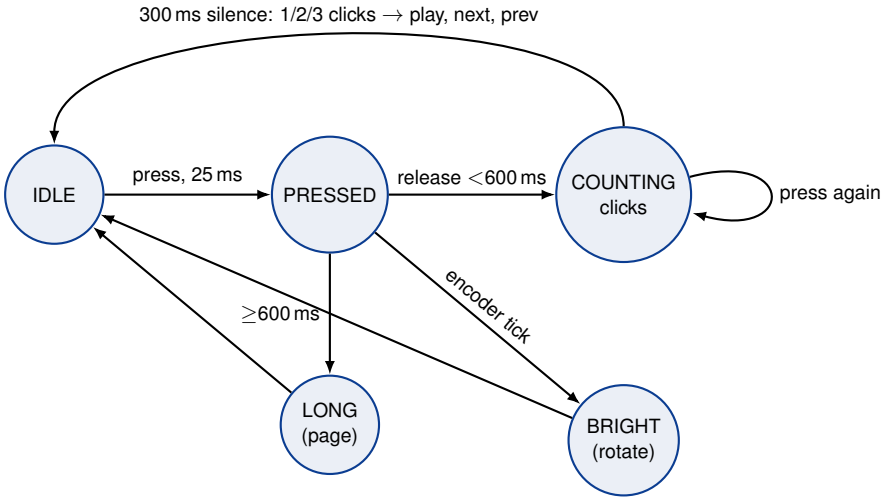


Figure 6: Button gesture state machine implemented in `handleButton()` (lines 539–570).

Four-button edition. `updated.ino` replaces the above with a small `DebouncedButton` struct and fires on the falling edge after 25 ms. Pin choices and the handler are shown in Listing 5. This variant has no local volume or brightness control (the constant `VOLUME_STEP` is declared but unused); volume is changed from the phone and mirrored back by `syncRemoteVolume()`.

Listing 5: Four-button debouncing (`updated.ino`, lines 516–548).

```
516 // Button debouncing & event handling
517 bool updateButton(DebouncedButton &btn) {
518   const uint32_t now = millis();
519   const bool raw = digitalRead(btn.pin);
520   if (raw != btn.lastRawState) {
521     btn.lastRawState = raw;
522     btn.lastDebounceTime = now;
523   }
524   if (raw != btn.stableState && (now - btn.lastDebounceTime) >= 25) {
525     btn.stableState = raw;
526     markActivity();
527     // Trigger action on press (falling edge because of INPUT_PULLUP to GND)
528     if (btn.stableState == LOW) {
529       return true;
530     }
531   }
532   return false;
```

```
}

void handleButtons() {
  if (updateButton(btnPlayPause)) {
    togglePlayback();
  }
  if (updateButton(btnNext)) {
    nextTrack();
  }
  if (updateButton(btnPrev)) {
    previousTrack();
  }
  if (updateButton(btnPage)) {
    switchPage();
  }
}
```

4.5 User interface

The UI has two pages (three in the button edition): *Now Playing* (title and artist marquees, progress bar, elapsed/total time, animated eight-bar equaliser) and *Info* (album, battery percent and voltage, volume/brightness, uptime). A top bar always shows a Bluetooth glyph, a connection indicator (pulsing ring while pairing, solid dot when linked), play/pause icon, volume and battery. Transient overlays – a volume bar and toast notices – are drawn on top for 1.3 s. A pairing animation of radiating rings is shown while disconnected.

The marquee (Listing 6) shows static text when it fits and otherwise scrolls at 32 px/s after a 1.5 s pause, drawing the string twice with a 28 px gap inside a clip window for a seamless loop.

Listing 6: Looping marquee (`main.ino`, lines 260–281).

```
// Looping marquee clipped to [x, x+width). Static text if it fits.
void drawMarquee(Marquee &m, const String &text, int x, int baselineY, int width, int top, int bottom) {
  if (m.text != text) {
    m.text = text;
    m.start = millis();
  }
  const int tw = textWidth(text);
  if (tw <= width) {
    u8g2.drawStr(x, baselineY, text.c_str());
    return;
  }
  const int gap = 28;
  const int cycle = tw + gap;
  const uint32_t pauseMs = 1500;
  const uint32_t scrollMs = uint32_t(cycle) * 1000UL / 32UL; // 32 px/s
  const uint32_t phase = (millis() - m.start) % (pauseMs + scrollMs);
  const int offset = phase < pauseMs ? 0 : int((phase - pauseMs) * 32UL / 1000UL);
  u8g2.setClipWindow(x, top, x + width, bottom);
  u8g2.drawStr(x - offset, baselineY, text.c_str());
  u8g2.drawStr(x - offset + cycle, baselineY, text.c_str());
  u8g2.setMaxClipWindow();
}
```

Bus budget. A 128×64 monochrome frame is $128 \times 64/8 = 1024$ bytes. At 400 kHz I²C each byte costs about nine bit-times, so a full-frame transfer needs roughly $1024 \times 9/400\,000 \approx 23\,\text{ms}$ plus addressing overhead. This fits inside the 40 ms active frame period, but leaves little slack – which explains the design choice to drop to 100 ms frames when idle.

4.6 Battery gauge

The cell voltage is read on ADC1 pin `IO34` (11 dB attenuation, 12-bit) through an assumed 1:2 divider (`BATTERY_DIVIDER_RATIO = 2`). A full-charge 4.2 V cell therefore presents 2.1 V and a 3.3 V cell 1.65 V, both inside the usable ADC range. Eight samples spaced 100 ms apart are averaged, then passed through

a first-order IIR filter with $\alpha = 0.15$:

$$y_k = y_{k-1} + \alpha\,(x_k - y_{k-1}), \qquad \tau \approx \frac{(1-\alpha)\,T_s}{\alpha} \approx \frac{0.85 \times 0.8\,\text{s}}{0.15} \approx 4.5\,\text{s}. \tag{2}$$

Percent is obtained by piecewise-linear interpolation over an 11-point LiPo discharge table (Figure 7); readings outside 2.5–4.35 V are reported as “no cell”. A 2 % hysteresis stops the displayed value from flickering.

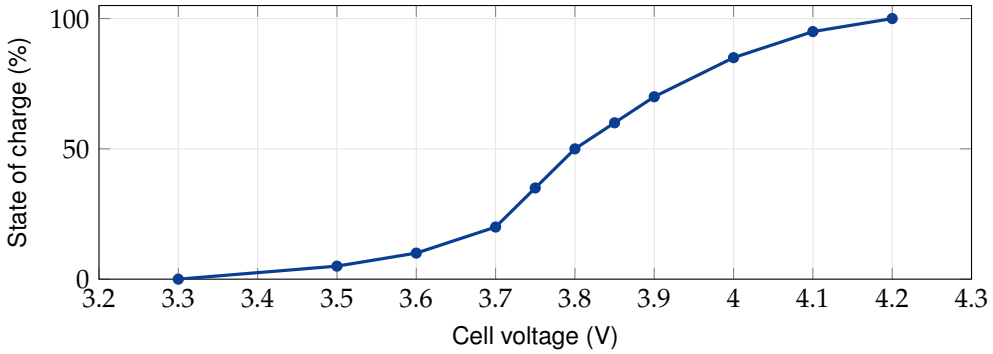


Figure 7: Voltage-to-percentage table in `estimateBatteryPercent()` (lines 215–226).

Listing 7: Battery sampling, filtering and hysteresis (`main.ino`, lines 228–251).

```
228 void updateBattery(bool immediate = false) {
229   static uint32_t lastSample = 0, totalMv = 0;
230   static uint8_t sampleCount = 0;
231   const uint32_t now = millis();
232   if (!immediate && now - lastSample < BATTERY_SAMPLE_MS) return;
233   lastSample = now;
234   totalMv += analogReadMilliVolts(BATTERY_ADC_PIN);
235   ++sampleCount;
236   if (!immediate && sampleCount < BATTERY_SAMPLES) return;
237   batteryAdcMv = (totalMv + sampleCount / 2) / sampleCount;
238   totalMv = 0;
239   sampleCount = 0;
240   const float measured = batteryAdcMv * BATTERY_DIVIDER_RATIO * BATTERY_CALIBRATION;
241   if (measured < 2500 || measured > 4350) {
242     filteredBatteryMv = 0;
243     batteryMv = 0;
244     batteryPercent = -1;
245   } else {
246     filteredBatteryMv = filteredBatteryMv == 0 ? measured : filteredBatteryMv + 0.15f * (measured -
          filteredBatteryMv);
247     batteryMv = uint32_t(filteredBatteryMv + 0.5f);
248     const int estimate = estimateBatteryPercent(filteredBatteryMv);
249     if (batteryPercent < 0 || abs(estimate - batteryPercent) >= 2 || estimate == 0 || estimate == 100)
          batteryPercent = estimate;
250   }
251 }
```

4.7 Display power management

After 30 s without input the contrast drops to 8 %; after 2 min *and* no playback the panel is put into power-save. Any input, track change or connection event counts as activity. This protects OLED lifetime and trims idle current.

4.8 Persistence

Volume and brightness are stored with the ESP32 `Preferences` (NVS) API. Writes are deferred until two seconds after the last change (`savePrefsIfNeeded`), so spinning the encoder produces one flash write rather than dozens – an important wear-levelling measure.

4.9 Concurrency and robustness

- Encoder ticks and metadata strings are protected by `portMUX` spin locks; critical sections are a few instructions long.
- Fixed-size `char` buffers with `strlcpy` prevent overflow from long tags.
- All hardware bring-up results are latched (`audioOK`, `oledOK`); the UI degrades gracefully ("Audio Error / Check I2S wiring"), and the OLED I^2C address is auto-detected (0x3C, then 0x3D).
- A compile-time switch `OLED_IS_SH1106` selects the 1.3" SH1106 driver (the common controller on such modules) versus SSD1306; the comment explains the 2-pixel shift symptom of a wrong choice.

5 Verification Plan

The repository contains no logged measurements, so this section defines the acceptance tests rather than reporting results. Each row should be filled in on the bench and the table committed alongside the firmware.

Table 5: Acceptance test matrix (results column intentionally empty).

ID	Test	Pass criterion	Result
T1	Pair from Android and iOS phones	Device appears as `ESP32_BT_Speaker`; stream starts	
T2	Power-cycle with phone in range	Auto-reconnect without user action	
T3	10 min continuous playback	No audible dropouts; no reset	
T4	Encoder rotation, both directions	4 % per detent, direction correct; no missed counts	
T5	1/2/3-click and long press	Play/pause, next, previous, page change	
T6	Press + rotate	Brightness changes; no click fired on release	
T7	Long titles with accents/emoji	Marquee scrolls; unsupported glyphs shown as ?	
T8	Battery sweep 4.2→3.3 V (bench supply)	Gauge monotonic; "no cell" outside 2.5–4.35 V	
T9	Idle 3 min	Dim at 30 s, display off at 120 s; wake on input	
T10	Reboot	Volume and brightness restored	

6 Design Review Findings and Recommendations

The following issues were found by comparing the schematics with the firmware. Severity: **H** will prevent correct operation, **M** likely to cause problems, **L** housekeeping.

7 Conclusion and Future Work

The firmware is a well-structured, readable embedded application: non-blocking services, ISR-safe encoder decoding, thread-safe metadata exchange, graceful degradation and wear-aware persistence. The principal risk to a working product lies in hardware documentation rather than code: the schematics and sketches disagree on pins and parts, and the power path is incomplete. Closing findings F1–F6 and running the test matrix would move the project from a prototype to a design that can be released as a PCB.

Roadmap. (1) unify the pin map; (2) KiCad ERC clean and PCB layout with an RF keep-out around the ESP32 antenna; (3) battery divider, regulator and protection; (4) optional SD/MP3 mode using the bundled libhelix; (5) power measurement per mode; (6) enclosure and acoustic tuning.

Table 6: Review findings.

ID	Sev.	Observation	Recommendation
F1	H	Schematic wires six buttons to `IO32,33,34,35,16,17`; `main.ino` uses `IO32,33,14` for the encoder and `updated.ino` uses `IO14,12,13,4`.	Make the schematic the single source of truth and generate the pin constants from one table.
F2	H	`IO34` is wired to a push-button in the schematic *and* used as the battery ADC in both sketches. `IO34/IO35` are input-only and have no internal pull-up, so `INPUT_PULLUP` cannot work on them.	Move the battery sense to another ADC1 pin or move the button; add external pull-ups for any button on `IO34/IO35`.
F3	M	Display symbol is a 0.91" 128×32 part (`ER-OLEDM0.91`); firmware drives a 1.3" 128×64 SH1106. Pinout of real modules is usually VCC/GND/SCL/SDA in a different order.	Replace the symbol with the actual module and verify pin order.
F4	M	The 15 net labels (`IO5`, `IO18`, ...) appear only on the peripheral side in the renderings; I found no matching labels at the ESP32 symbol. In KiCad this would leave the nets unconnected (single-label ERC warning).	Open in KiCad, run ERC, and add the matching labels (or wires) at the ESP32 pins.
F5	M	No 3.3 V regulator, no battery-to-5 V boost and no battery voltage divider are drawn; TP4056 output is not connected to the loads.	Add a boost/load-sharing stage, a 3.3 V LDO and the 100 kΩ/100 kΩ divider assumed by the firmware.
F6	M	`IO12` (used by the button edition) is a boot-strapping pin; holding it high at reset selects 1.8 V flash and can prevent boot.	Avoid `IO12`; use non-strapping pins such as `IO13`, `IO14`, `IO16` or `IO17`.
F7	L	SD socket is drawn but no firmware uses it; the repository name suggests local MP3 playback.	Either rename the project or add an SD player mode (libhelix is already bundled).
F8	L	"Live Lyrics" page in `updated.ino` only shows "Now playing : title by artist".	Rename the page or integrate a real lyrics source.
F9	L	Capacitor C3 is drawn as 0.5 µF, a non-standard value; `README` says PCM5102A is "much costlier".	Use 1 µF or the module vendor's recommended value.

Build notes

Build steps. (1) Install the Arduino IDE and the ESP32 board package. (2) Install U8g2 from the Library Manager. (3) Use *Sketch* → *Include Library* → *Add .ZIP* for the bundled ESP32-A2DP and arduino-audio-tools archives (and libhelix), one at a time, then restart the IDE. (4) Select "ESP32 Dev Module", a partition scheme with enough application space for the Bluetooth stack (e.g. "Huge APP"), and upload.

Reference. MAX98357A, PCM5102A and TP4056 datasheets; ESP32-WROOM-32 datasheet (strapping pins); P. Schatzmann, *ESP32-A2DP* and *arduino-audio-tools*; O. Kraus, *U8g2*.

Appendix – Full Schematics

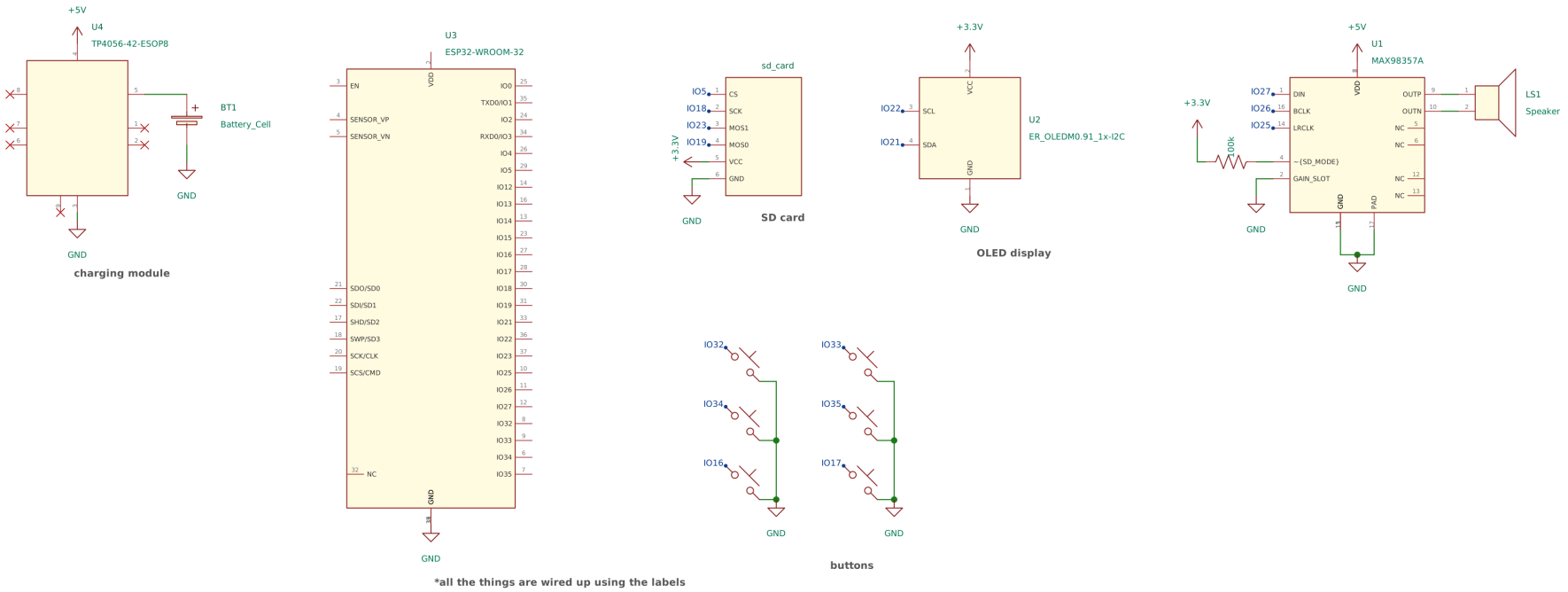


Figure 8: Speaker variant (`mp3.kicad_sch`), rendered from the KiCad file.

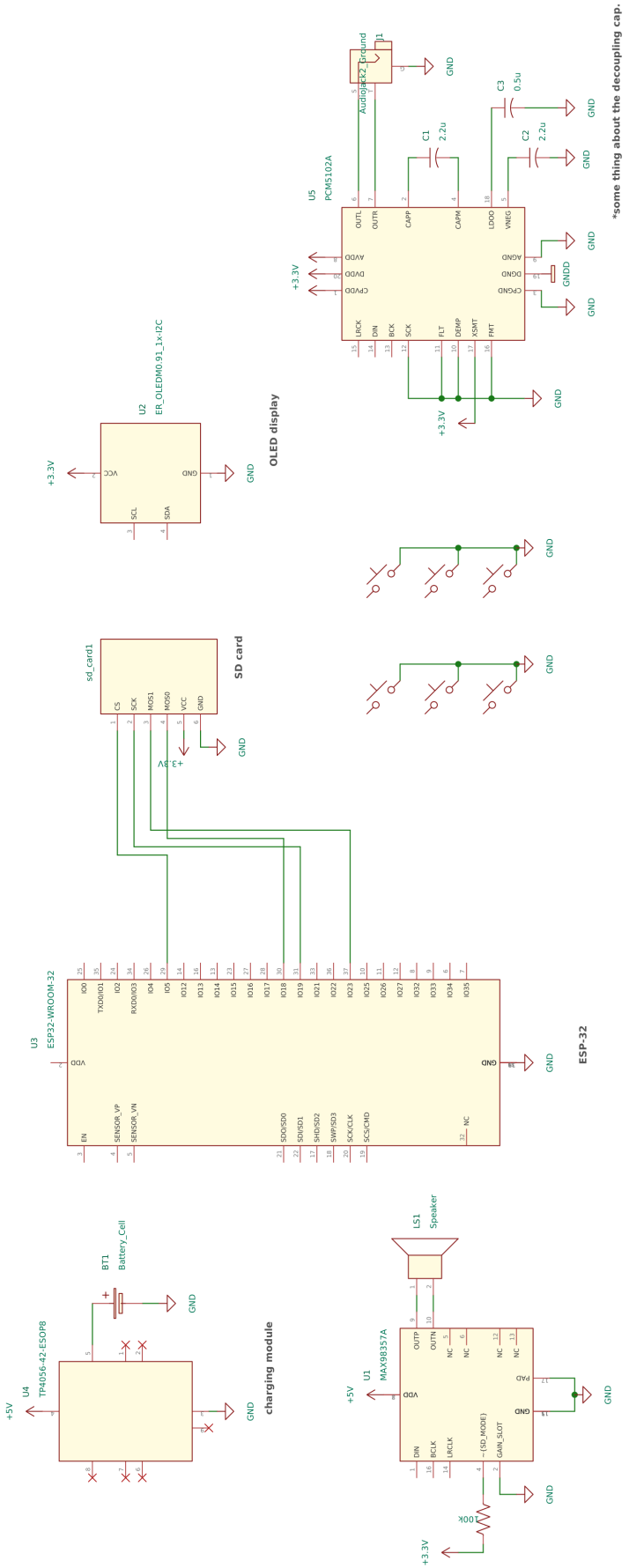


Figure 9: Headphone variant (`with headphone.kicad_sch`), rendered from the KiCad file.